

Sync Team Project

GeoAnalytics Planner Dashboard

Laboratory of advanced programming

Alessandro Savo, 2021372

Leonardo Santucci, 2282707

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Business Problem

- **Context:** The project helps a federation of three companies plan visits for their sales representatives.
- **Data:** We start from an Excel file containing customers, delivery points, sales agents, and revenue data.
- **The Challenge:** Sales representatives have limited time. The problem is deciding **which customers to visit**, in **which order**, and on **which days**.
- **The Goal:** Cover as much relevant revenue as possible within the available working time.
- **What-if Analysis:** The user can easily compare different scenarios by changing the number of available days and the duration of each visit.



Main User Stories (1/2)

- **Upload & Reuse**

"As a Business Analyst, I want to upload an Excel file, so that I can work with the latest data extracted from the ERP."

- **Company Selection**

"As a Sales Manager, I want to select a company, so that I can analyze only the customers and revenue related to that company."

- **Customer Map**

"As a Sales Manager, I want to see the customers of the selected company on a map, so that I can understand their geographic distribution."



Main User Stories (2/2)

- **Start Date & Working Days**

"As a Sales Manager, I want to choose the starting date of the visit campaign, so that I can plan visits for a specific commercial period."

- **Suggested Visit Sequence**

"As a Sales Manager, I want the system to create a suggested sequence of customer visits, so that I can organize the working day."

- **Recoverable Revenue**

"As a Sales Manager, I want to see the estimated recoverable revenue of the proposed plan, so that I can understand the economic value."

- **What-if Analysis**

"As a Sales Manager, I want to change the number of available days and run the analysis again, so that I can compare different scenarios."



Non-Functional Requirements

- **Usability**

The application should be easy to use and understandable without specific technical knowledge.

- **Reusability**

The application should be reusable with future ERP files having the same logical structure.

- **Portability**

The application should be deployable on another computer using Docker and Docker Compose.

- **Accuracy & Robustness**

Provide clear error messages for invalid files and use real geographic coordinates for customer locations.



From Requirements to Interface

These mockups were used to define the structure of the dashboard before implementation.

Mockup 1

Upload and Scenario Setup

Sales Visit Optimizer

ERP Excel File

Company
WINES ▼

Campaign Start Date

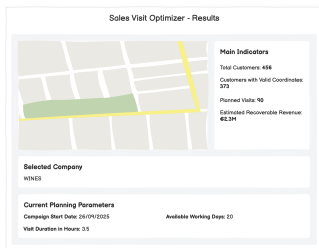
Available Working Days

Visit Duration in Hours

Run Analysis

Mockup 2

Results, Map and KPIs



Mockup 3

Suggested Visit Plan

Suggested Visit Plan

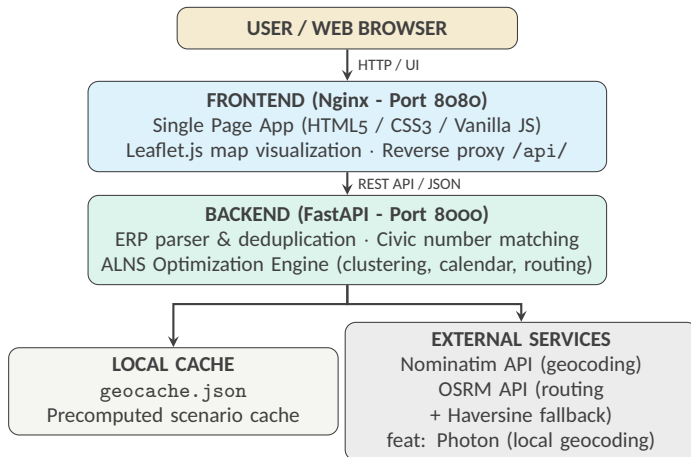
Selected Company
WINES

Visit Agenda

DATE	DAY	TIME	CUSTOMER	CITY	ADDRESS	ESTIMATED REVENUE
01/10/2023	Wednesday	09:00 - 11:00	Customer A	Rome	Via Salaria 56	€1,000
01/10/2023	Wednesday	11:30-13:30	Customer B	Rome	Via Bove 34	€7,500
02/10/2023	Thursday	14:30-16:30	Customer C	Guidonia	Via Roma 144	€3,200
03/10/2023	Friday	9:00-11:00	Customer D	Rome	Via Tiburtina 630	€4,100



Software Architecture





Docker Deployment

Docker

Packages the application and its dependencies.

Docker Compose

Orchestrates multi-container lifecycle.

Frontend: Port 8080 (Web UI)

Backend: Port 8000 (REST API)

`docker-compose.yml`

```
backend:
  build: ./backend
  ports: ["8000:8000"]

frontend:
  build: ./frontend
  ports: ["8080:80"]
  depends_on: [backend]
```

Separation of Concerns

Separate host ports **isolate** the end-user Web UI from direct Swagger/REST API debugging.



Data Processing & Deduplication

The same physical location often appears multiple times in the ERP dataset (e.g. multiple invoice or delivery rows).

ERP Deduplication Example (Selected Company: WINES)

Customer A – Via Roma 10 – Row 1 (WINES): EUR 600

Customer A – Via Roma 10 – Row 2 (WINES): EUR 400

↓ **GROUP BY PHYSICAL LOCATION** ↓

Customer A – Via Roma 10 – Total WINES: EUR 1,000 (*1 visit planned*)

1. **Flexible parser:** Handles arbitrary column order and discovers new companies.
2. **Deduplicate:** Groups duplicate ERP rows for the chosen company.
3. **Geocode:** Converts unique delivery addresses into coordinates.
4. **Travel matrix:** Computes driving times with OSRM and Haversine.



Optimization Problem

Finding the exact global optimum is **NP-hard** (Team Orienteering with Time Windows).
An exhaustive search is computationally intractable, requiring **heuristic optimization**.

INPUTS

- ERP Customers & Revenues
- Starting Depot location
- Available business days (N)
- Target visit duration (3h–4h)
- OSRM travel duration matrix

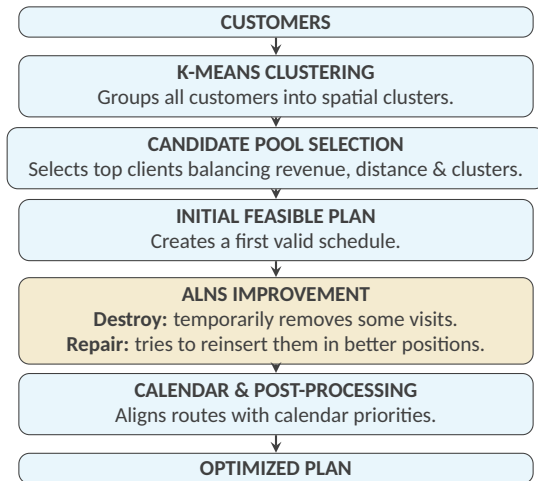
CONSTRAINTS

- **Calendar:** Mon–Fri (skip holidays)
- **Work budget:** max 8h/day (09–18)
- **Lunch slot:** flexible 1h window
- **Uniqueness:** at most 1 visit/client
- **Transfers:** multi-day trips for remote clients

Objective: Maximize total recovered revenue while guaranteeing feasibility.



How the Optimizer Works





ALNS - Adaptive Large Neighborhood Search

Simplified logic — not the actual source code

```
best = current = initial_plan

for each iteration:
    candidate = repair(destroy(current))

    if candidate is better than current:
        current = candidate
        if current is better than best:
            best = current

    elif accept_with_probability(p):
        current = candidate

return best
```

DESTROY

Remove some visits (random/cluster).

REPAIR

Reinsert visits in optimal positions.

ACCEPT

Keep improving plans or explore with probability p .

BEST

Track the best plan found so far.



ALNS - Destroy

alns.py (Random Destroy)

```
num_remove = max(1, int(len(
    route.client_indices) *
    random.uniform(0.1, 0.2)))

indices_to_remove = random.sample(
    range(len(route.client_indices)),
    num_remove)

for idx in sorted(indices_to_remove,
                  reverse=True):
    client_idx = \
        route.client_indices.pop(idx)
```

1. **Decide removal quota** (10–20% of day's visits).
2. **Choose visits** via random sampling.
3. **Remove visits** to open available capacity.

***Dual strategy:** Alternates with **cluster destroy** to reshape entire geographic zones.*

Why?

Escapes local optima by freeing time slots to explore more profitable visit combinations.



ALNS - Repair

alns.py & construction.py

```
# Best insertion scoring logic
for day, pos in feasible_slots:
    extra = new_r.travel - r.travel
    score = c.revenue / max(1, extra)

    if shares_cluster:
        score *= 1.10 # Compactness

    if score > best_score:
        best_slot = (day, pos)
```

Goal:

Rebuilds dense, high-yield routes by prioritizing clients that bring maximum revenue for minimal extra driving time.

1. Candidate Selection

Removed visits + unscheduled accounts.

2. Feasibility Filter

Enforces work hours, lunch, and transfers.

3. Greedy Slot Assignment

Maximizes Δ Revenue/ Δ Travel Time.



Testing

Test Execution

```
$ python3 -m pytest -q source/test
.....
6 passed
```

1. Daily capacity

Four 2-hour visits fit in one working day when travel time is zero.

2. Weekend rule

Saturday is moved to the next working day.

3. National holidays

Christmas and Easter Monday are skipped.

4. Transfer logic

A nearby client is not treated as a transfer, while a distant client is.



Conclusions

- **Data Robustness:** Preprocessing and geocaching are critical to handle messy real-world addresses.
- **Spatial Intelligence:** K-Means clustering prevents route scatter, ensuring compact and efficient itineraries.
- **Metaheuristic Efficiency:** ALNS solves an NP-hard problem, maximizing revenue in seconds without brute-force.
- **Operational Realism:** Strictly enforces practical business constraints (work hours, lunch breaks, multi-day trips).
- **Production Readiness:** Dockerized microservices ensure fast, reliable, and reproducible deployment.



Live Demo

Now let's see the Demo in action!

Live Application:

<http://localhost:8080>